

## Problem 1

1A) Two processes are actively running on a single-CPU, X86-32 Linux system. By default all programs are compiled such that their text regions start at the same virtual address 0x08040000. Describe what happens in terms of memory performance at context switch time if (case 1) the CPU doesn't have the PCID feature vs (case 2) it does.

Case 1:

The kernel simply writes the %cr3 register with the physical address of the PGD representing the new process. Then, the CPU hardware invalidates (flushes) all TLB entries, except those where the cached PTE has a special G (Global) flag set.

Case 2:

The kernel modifies writes the %cr3 register with both the physical address of the PGD and the new PCID. In this case, the TLB is not flushed, and all entries remain cached. The CPU automatically uses the current PCID when looking up translations. Since both processes can have entries for the same virtual address with different PCIDs, only entries matching the current PCID are considered for hits. This means better performance as there are far fewer TLB misses requiring costly page table walks.

1B) On X86-32 Linux systems, the initial stack region is one page long and starts at 0xBFFFFFF00, i.e. the vm\_start field of the associated vm\_area\_struct is 0xBFFFFFF00 and vm\_end is 0xBFFFFFFF. After the process has been running for a while, a page fault is incurred. The faulting address is 0xBFFFEFFC. What happens? be sure to discuss in terms of the BSD kernel as opposed to Linux.

The fault region is close to the stack region.

BSD: "If the memory region just above the faulted address has the GROWSDOWN property, then this stack region is expanded by one page toward low memory addresses ("down")."

Linux: The kernel manages memory mapping through vm\_area\_struct, and stack region with the VM\_GROWSDOWN flag can expand downward when a page fault occurs near the stack boundary.

[Both] (Assuming the expansion will not cause a collision with any other regions or exceed the stack region) Then, the kernel expands the stack region from 0xBFFFFFF00~0xBFFFFFFF to 0xBFFFE000~ 0xBFFFFFFF as  $0xBFFFFFF00 - 0x1000 = 0xBFFFE000$ . After the expansion, the kernel verifies that the access type (rwx) is valid for this area and completes the fault handling.

1C) We've been playing CounterStrike where the executable file is located via a network filesystem on a remote server. Everything has been working fine. But unbeknownst to us, the network cable to file server got unplugged. Subsequently, we explore a feature of the game that we hadn't previously used. What do we expect will happen and why? Hint: the M-8.

The process receives a SIGBUS signal or an I/O error.

The new feature's code was not loaded into the memory, so attempt to execute it causes a page fault. Then, the page fault handler tries to read the code from the remote network and realizes I/O failure.

1D) A particular executable file /home/student/a.out is being run as process 6767. The file has mode 755 (rwxr-xr-x). Process 555 does `open("/home/student/a.out", O_RDWR)`. Both 555 and 6767 are the same uid. What happens and why?

Linux prevents opening an executable file for writing when it is currently being executed. Even though both processes have the same uid and the file permissions allow writing, the kernel causes the `open()` call to fail for process 555 because the same file is being executed by process 6767.

1E) On a 64-bit X86 system, the first PDE (index 0) of the PGD is all-0. This means that virtual memory from 0x0000 0000 0000 0000 to \_\_\_\_\_ is unmapped. Fill in the blank, and show work!

Since PGD entry 0 is all zeros, we only need to consider 0-38 index bits out of 64 bits. Thus  $2^{39} = 512 \text{ GB}$ .

Answer: 0x0000 007F FFFF FFFF